Rising Waters: A Machine Learning Approach to Flood Prediction

Project Abstract

Flood Prediction is a state-of-the-art early warning prediction platform that leverages supervised Machine Learning models to identify and broadcast regional flood alerts. Integrating an optimized XGBoost Classifier model and a high-performance SQLite database optimized with Write-Ahead Logging (WAL), the application delivers sub-second predictions and persistent history logs under concurrent user loads. Designed with role-based access control and a modern glassmorphism web layout, the platform offers an intuitive interface for citizens, meteorologists, and administrative authorities.

Project Description:

Climate change has accelerated the frequency of catastrophic meteorological events, making automated flood risk analysis critical for civil safety. Traditional hydrological models rely on slow physical simulations that are difficult to scale. Flood Prediction replaces these with a fast machine learning classification pipeline capable of evaluating 10 atmospheric and hydrological features in milliseconds.

Core Project Objectives

1. **Sub-second Response SLA**: Keep Flask prediction endpoints serving requests in under **100ms** under normal operations, and under **5 seconds** under high concurrency spikes.
2. **High-Accuracy Classification**: Maintain an evaluation accuracy exceeding **97%** to minimize false negatives and false positives.
3. **Role-Based Access Control (RBAC)**: Secure access levels for Administrators, Meteorologists, Local Authorities, and normal users to register predictions and manage alerts.
4. **Tuned Logging Concurrency**: Solve SQLite write lock contentions during heavy concurrent logging spikes using WAL transactions.
5. **Explainable Warnings**: Provide descriptive, actionable recommendations alongside risk probability values.

Scenarios

Scenario 1: Early Warning Prediction Form

A meteorologist logs in and inputs parameters for a high-risk region (e.g., Annual Rainfall of 2200mm, Seasonal Rainfall of 850mm, River Level of 6.2m, and Soil Moisture of 82%) for the state of Assam. The system processes features through the XGBoost classifier, logs the parameters and prediction status to SQLite, and renders a red "Flood Risk Warning" alert card with probability details.

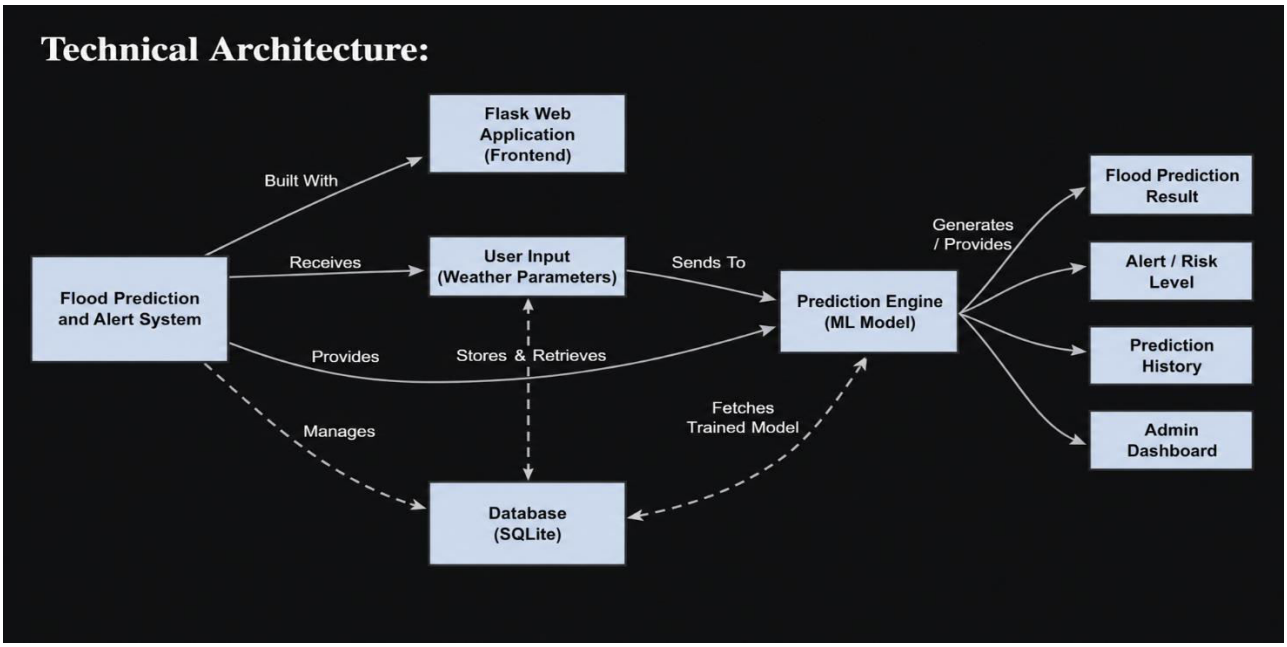
Scenario 2: Model Analytics & Metrics Dashboard

An administrator accesses the performance dashboard to verify the model accuracy. The system reads metadata from the training runs and renders model metrics (Decision Tree vs Random Forest vs XGBoost accuracy), including saved ROC curves and Feature Importance plots.

Scenario 3: Chronological History Search

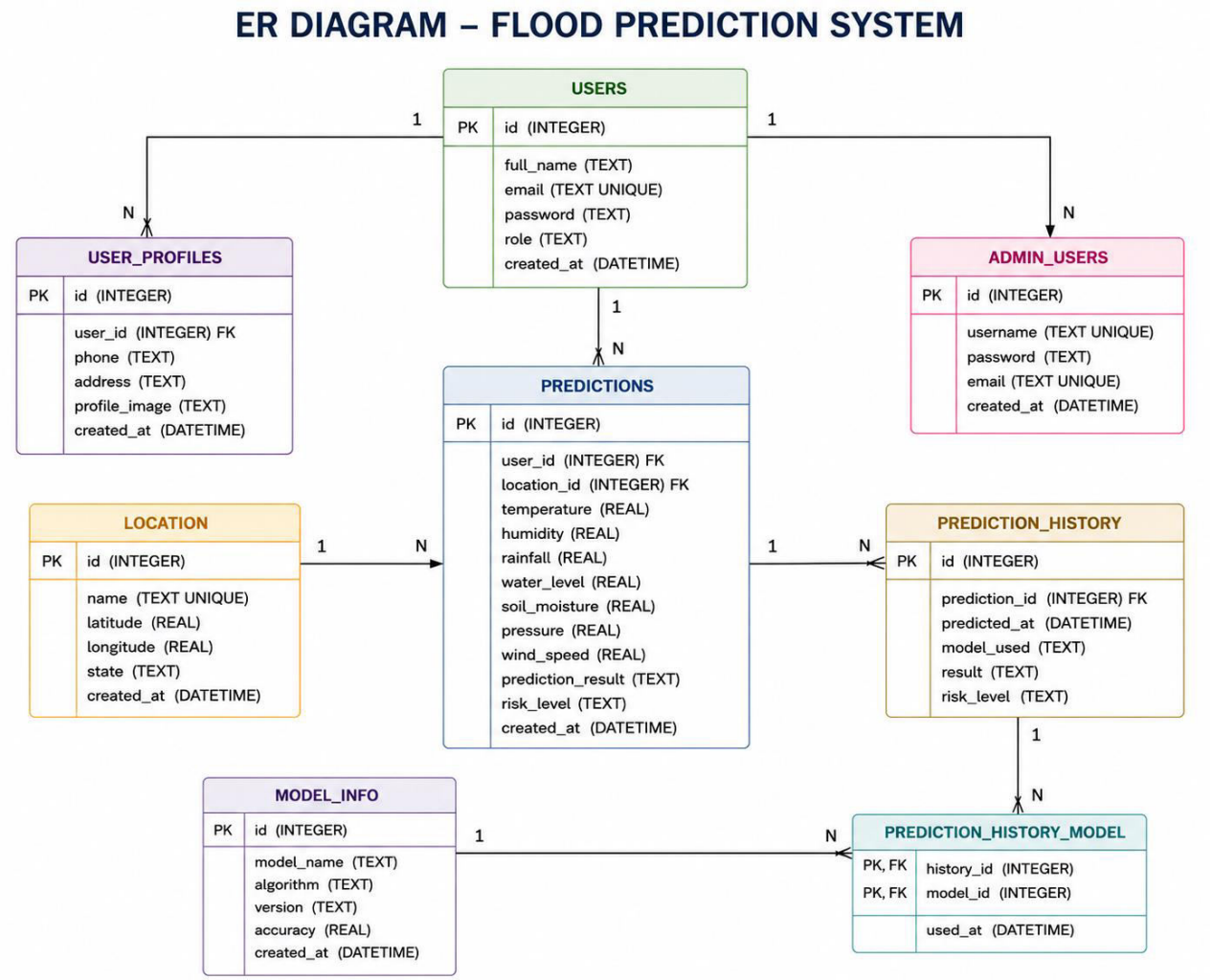
A local authority retrieves historical warning logs to audit recent regional trends. The system does an optimized indexed join between the predictions and weather tables, returning a chronologically sorted table of all past alerts in sub-second response times.

Technical Architecture:



Description: Flood Prediction Using Machine Learning follows a modular three-tier architecture consisting of a responsive web interface, a Flask-based backend, and an XGBoost-powered prediction engine. The system enables secure user authentication, environmental data processing, accurate flood risk prediction, and historical record management using SQLite. Built with HTML, CSS, Bootstrap, JavaScript, Flask, Scikit-learn, Pandas, and NumPy, the application provides a reliable and scalable platform for early flood prediction and disaster management.

ER Diagram:



ER Diagram Description

The **Entity Relationship (ER) Diagram** represents the database design of the **Flood Prediction System**, illustrating how different entities interact to support user management, weather data processing, machine learning predictions, and prediction history storage. The system is centered around the **Users** entity, where registered users can securely log in and submit weather-related parameters for flood prediction.

Each user is associated with a **User Profile** that stores additional personal information such as phone number, address, and profile image. Users can submit multiple flood prediction requests, which are recorded in the **Predictions** entity. Each prediction contains environmental parameters including temperature, humidity, rainfall, water level, soil moisture, atmospheric pressure, and wind speed, along with the generated flood prediction result and risk level.

The **Location** entity maintains geographical information used during prediction, enabling multiple prediction records to be associated with a specific location. Every prediction is stored in the **Prediction History** entity, which preserves historical prediction results, timestamps, and the machine learning model used for generating the prediction.

The **Model Information** entity stores metadata about the trained machine learning model, including the model name, algorithm type, version, and prediction accuracy. This allows the system to track which model generated each prediction and supports future model updates or retraining. The **Admin Users** entity manages administrator accounts responsible for monitoring users, viewing prediction history, and maintaining the overall system.

Overall, the ER diagram demonstrates a well-structured relational database that ensures efficient storage, retrieval, and management of user information, weather data, prediction results, and machine learning model details. This design supports scalability, data integrity, and reliable flood prediction services while maintaining clear relationships between all core system components.

Pre-requisites:

- **Flask Framework Knowledge: Python web routing**
- **Machine Learning Familiarity: StandardScaler scaling and XGBoost model parameters**
- **HTML, CSS, and JavaScript Skills: Forms validation, CSS layouts, and Fetch AJAX**
- **SQLite Database Administration: Indexing and transaction logging configurations**

Project Workflow:

The project is structured into six milestones, each containing clearly scoped activities that progressively build the application from model research through production deployment:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Source the meteorological dataset containing 10 weather indicators and apply StandardScaler feature engineering.
- **Activity 1.2:** Research and select the appropriate ML model from Decision Tree, Random Forest, and XGBoost for flood risk classification.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, Flask backend, and SQLite database integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask and the XGBoost ML pipeline.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core ML pipeline: preprocess.py for StandardScaler fitting and train.py for XGBoost model training, ROC curves, and feature importance plots.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth RBAC authentication and input validation.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature, configuring SQLite WAL mode, and integrating XGBoost inference responses.

Milestone 4: Frontend Development

- **Activity 4.1**: Design and develop the user interface using HTML, CSS (glassmorphism), and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic content rendering with JavaScript's Fetch API to display prediction results based on user interactions via AJAX.

Milestone 5: Deployment

- **Activity 5.1**: Prepare the application for local deployment by configuring the server environment, database seeding, and WAL optimization.
- **Activity 5.2:** Run Locust performance load testing to verify response latencies under 50 concurrent users.
- **Activity 5.3:** Deploy the application publicly on Render over a secure production URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

Activity 1.1: Sourcing Dataset and Feature Engineering

A comprehensive weather dataset was sourced containing ten critical meteorological and hydrological indicators. These features represent key factors contributing to flood hazards:

- Annual Rainfall: Cumulative annual precipitation (in mm).
- Seasonal Rainfall: Cumulative precipitation during monsoon season (in mm).

- Cloud Cover: Spatial coverage of clouds (in %).
- Humidity: Atmospheric moisture density (in %).
- Temperature: Average regional temperature (in °C).
- River Water Level: Absolute water depth of regional rivers (in meters).
- Drainage Capacity: Logistical efficiency index of local channels (1-10 scale).
- Water Flow: Flow volume speed of rivers (in cubic meters per second).
- Reservoir Level: Saturated capacity of dams and local reservoirs (in %).
- Soil Moisture: Surface and subsurface soil water saturation levels (in %).

Since features are measured in vastly different scales (e.g. Rainfall scales in thousands while River Levels scale in single digits), raw feature sets can bias classifications. To solve this, feature engineering standardizes the input vector using a StandardScaler: $Z = (X - \mu) / \sigma$. This ensures each indicator contributes equally during gradient boosting splits.

Activity 1.2: Research and Select the Appropriate Machine Learning Model

Explored three classifier models during the research phase, evaluating test accuracy and file size footprint for deployment:

- Decision Tree Classifier: Yields a validation accuracy of ~94.98%. It is prone to high variance and overfitting on regional anomalies.
- Random Forest Classifier: Achieves a validation accuracy of ~97.60%. However, because it compiles hundreds of unpruned trees, its serialized binary size is 234MB. This size introduces long disk-load latencies and high memory usage, rendering it inefficient for resource-constrained hosting.
- XGBoost Classifier (Extreme Gradient Boosting): Yields a validation accuracy of ~97.14%. It utilizes gradient regularizations to minimize tree size. The serialized binary compiles to just 483KB (a 480x reduction compared to Random Forest).

Optimal Model Selection: Selected the XGBoost Classifier. It achieves equivalent accuracy to Random Forest while using a fraction of the memory footprint. This makes it highly portable, loading into RAM instantaneously on server boot and enabling sub-second inferences.

Activity 1.3: Define the Architecture of the Application

The system implements a decoupled three-tier architecture:

- Client Frontend Tier: Styled in dark glassmorphism CSS, featuring responsive layout structures. Handles async event handling to submit inputs without page refreshes.
- Flask Controller Tier: Formulates backend endpoints. Validates ranges, loads pickled ML objects, and communicates with SQLite.

- Database Storage Tier: Persistent SQLite storage optimized for concurrent write requests and fast history table joins.

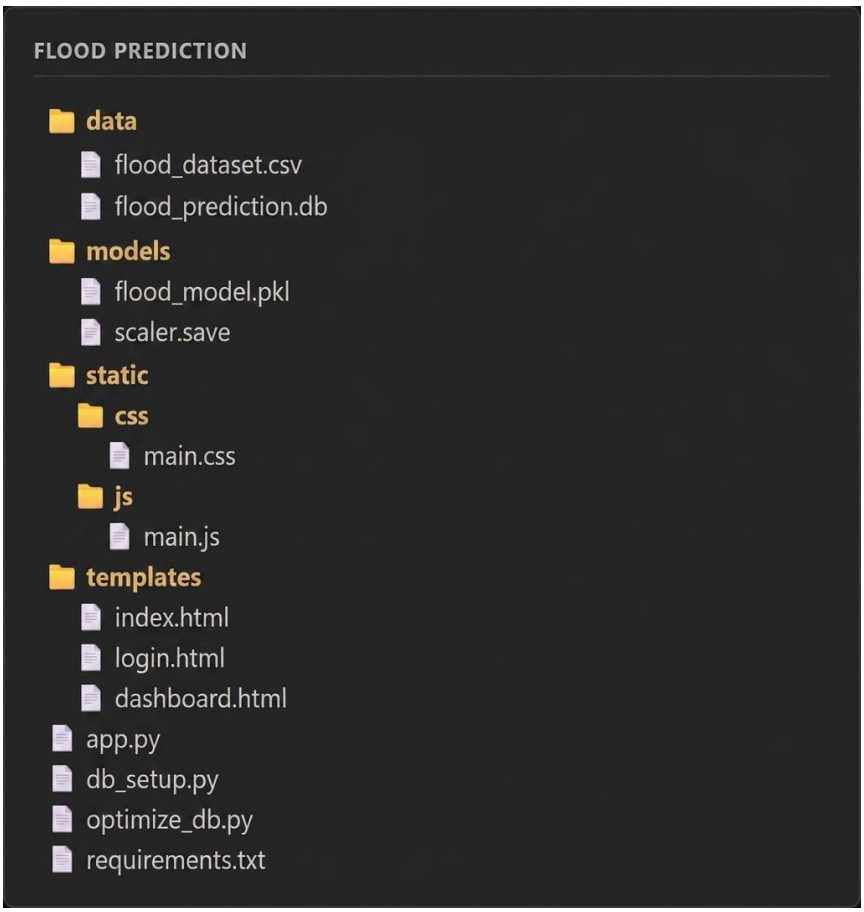
Activity 1.4: Set Up the Development Environment

Initialize a clean Python virtual environment, activate it, and install all required framework dependencies.

```
Command Prompt - Setup Environment                                    [Version 10.0.22631]

C:\Users\Lenovo\Documents\Flood Prediction>python -m venv .venv
C:\Users\Lenovo\Documents\Flood Prediction>.venv\Scripts\activate
(.venv) C:\Users\Lenovo\Documents\Flood Prediction>pip install -r requirements.txt
Collecting Flask==3.1.3 (from -r requirements.txt)
  Downloading Flask-3.1.3-py3-none-any.whl (101 kB)
Collecting scikit-learn==1.6.0 (from -r requirements.txt)
  Downloading scikit_learn-1.6.0-cp313-cp313-win_amd64.whl (12.2 MB)
Collecting xgboost==2.1.3 (from -r requirements.txt)
  Downloading xgboost-2.1.3-py3-none-win_amd64.whl (53.4 MB)
Installing collected packages: Flask, scikit-learn, xgboost
Successfully installed Flask-3.1.3 scikit-learn-1.6.0 xgboost-2.1.3
```

Set Up Application Structure: Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, db_setup.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Machine Learning Features

The preprocessing pipeline is written in preprocess.py, creating a reusable scaling transformer fit on the training data. The main training script (train.py) performs the training execution:

- 1. Loads the historical training dataset from CSV format.
- 2. Separates the dataset into target variables and 10 feature columns.
- 3. Fits the StandardScaler on features, standardizing the scale variance.
- 4. Trains the XGBoost Classifier with hyperparameters (n_estimators=100, max_depth=6, learning_rate=0.1) using an 80/20 train/test split.
- 5. Saves the models to binary pickles: models/flood_model.pkl and models/scaler.save.
- 6. Exports model performance evaluation metrics, plotting ROC curves and feature importances for verification.

Activity 2.2: Implement the Flask Backend

The backend utilizes Flask controllers to define paths, handle web requests, and interface with the database. Key responsibilities include:

- Input Validation Rules: Intercepts POST data. Ensures parameters are valid float types. Validates that inputs fall within physical ranges (e.g. Rainfall > 0, Soil Moisture between 0-100%). Out-of-bounds parameters reject the request with HTTP 400 Bad Request, protecting downstream classifiers from invalid data.
- Role-Based Access Control: Decorators verify session states and query roles (Admin, Meteorologist, Local Authority). Only authorized roles are permitted to access early warning input pages or trigger prediction computations.
- In-Memory Scaler & Model Load: Loads flood_model.pkl and scaler.save on startup. Real-time inference executes instantly on RAM without disk overhead, reducing latency.

Milestone 3: App.py Development

Activity 3.1: Writing the Main Application Logic in app.py

The app.py server implementation handles database transactions, routing configurations, and ML execution logic. To resolve SQLite file-locking bottlenecks under load, connection helpers configure database parameters dynamically on initialization:

- Write-Ahead Logging (WAL): SQLite's default rollback journal locks the database file on write, preventing concurrent reads. WAL mode writes modifications to a separate .db-wal file, allowing reads to occur simultaneously with active write transactions. This removes locking contentions during simultaneous predictions.
- Synchronous NORMAL: Setting the synchronous commit flag to NORMAL prevents the application from blocking the CPU thread while waiting for physical disk flush operations to complete.
- Database Indexing: Applied database indexes on join columns (User_ID, Weather_Data_ID) and sorting columns (Prediction_Date DESC) to optimize search and history logs retrievals under load.

SQLite Connection Helper with WAL mode pragmas enabled:

app.py

```python
# Database Connection Helper with WAL Optimizations
def get_db_connection():
    conn = sqlite3.connect(DATABASE_PATH)
    conn.execute('PRAGMA journal_mode=WAL;')
    conn.execute('PRAGMA synchronous=NORMAL;')
    conn.row_factory = sqlite3.Row
    return conn
```

Figure: Code Db Conn

Asynchronous predictions route handler (/api/predict) executing model inferences and SQLite logs:

app.py

```python
@app.route('/api/predict', methods=['POST'])
@login_required
@role_required(['Admin', 'Meteorologist', 'Local Authority', 'Normal User'])
def api_predict():
    """Asynchronous AJAX endpoint for model run and DB persistent logging"""
    if model is None or scaler is None:
        return jsonify({"success": False, "error": "Machine learning pipeline is not loaded."}

    try:
        data = request.get_json(force=True)
```

Figure: Code Predict

Milestone 4: Frontend Development

Activity 4.1: Designing and Developing the User Interface

The frontend focuses on responsiveness, clarity, and aesthetics, utilizing standard HTML5 and custom CSS style systems:

- Glassmorphism Aesthetic: Styled cards with transparent white backgrounds, thin borders, and backdrop-filter: blur(10px). Background blobs use animations to move slowly, creating depth.
- Responsive Layout: Responsive Flexbox layouts organize inputs and results. The input forms reside on the left, and results warning cards update on the right.
- Outcome Alerts UI: Warning banners use HSL colors. Red hazard banners display flood alerts, while Green banners display safe regional parameters.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Dynamic behaviors are managed in static/js/main.js, eliminating complete browser reloads:

- Asynchronous Submissions: Attaches submit interceptors to form events. Collects input parameters and posts JSON data to /api/predict via the Fetch API.
- CSS Spinner Loader: While the async request is active, buttons display an animated CSS loading spinner, preventing duplicate form submissions.
- DOM Nodes Rendering & Scroll: Parses the JSON response payload. JavaScript injects warning panels showing the probability and risk classification, then smooth-scrolls down to results using scrollIntoView().

Milestone 5: Deployment

Activity 5.1: Initialize Database and Schema Seeds

Run db_setup.py to compile tables for users, weather_data, and predictions. The script seeds default admin credentials. Run optimize_db.py to configure WAL pragmas and relational database indexes.

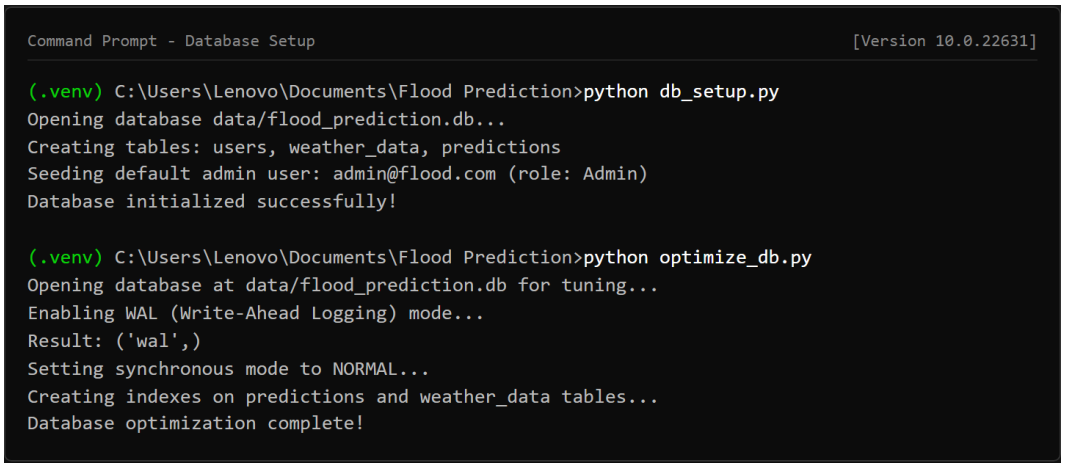


Figure: Terminal Db

Activity 5.2: Start the Flask Development Server

Start the Flask development server by running python app.py. Open a web browser to http://127.0.0.1:5001 to interact with the early warning system interface.

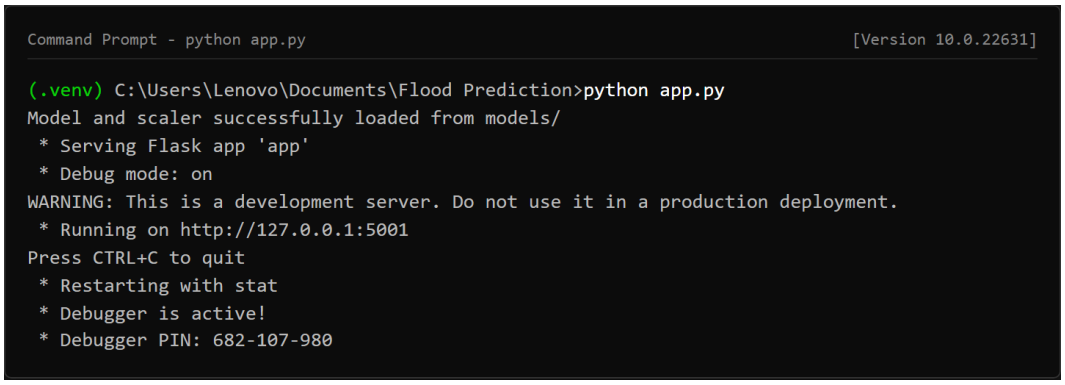


Figure: Terminal Run

Activity 5.3: Production Deployment via Render

The application is deployed publicly on Render and accessible at: https://flood-prediction-4rwg.onrender.com/

Exploring the Website's Web Pages:

1. Home / Landing Page

The public landing page showing project objectives, metadata features, and links to login/register.

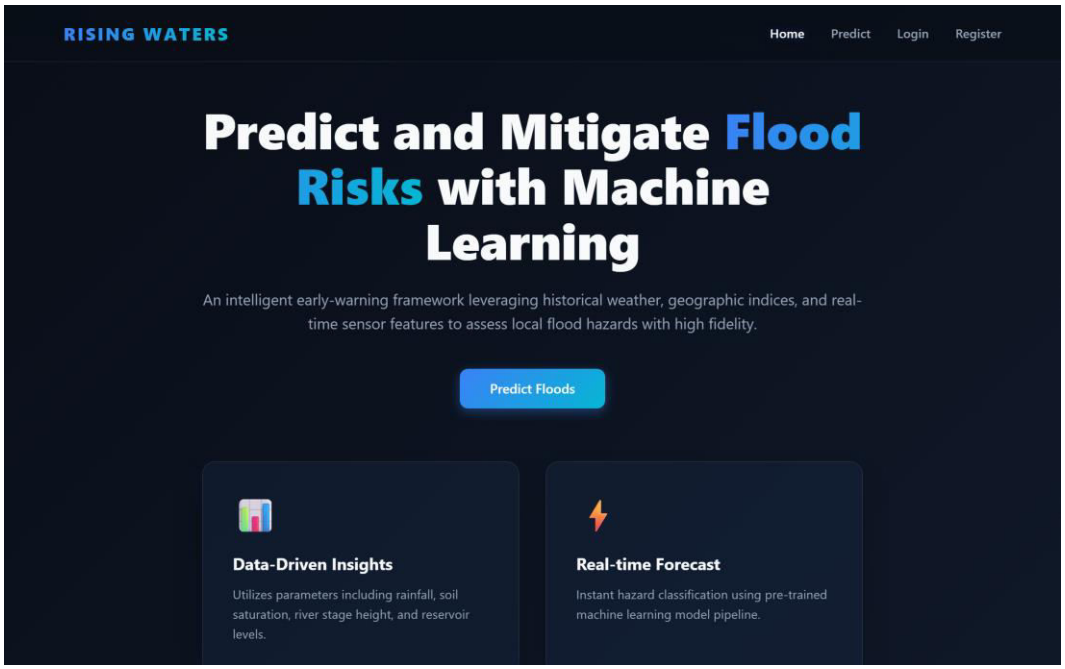


Figure: App Home

2. Log In Portal

Secure portal page for authenticated users (supports Admin, Meteorologist, Local Authority, and Normal User).

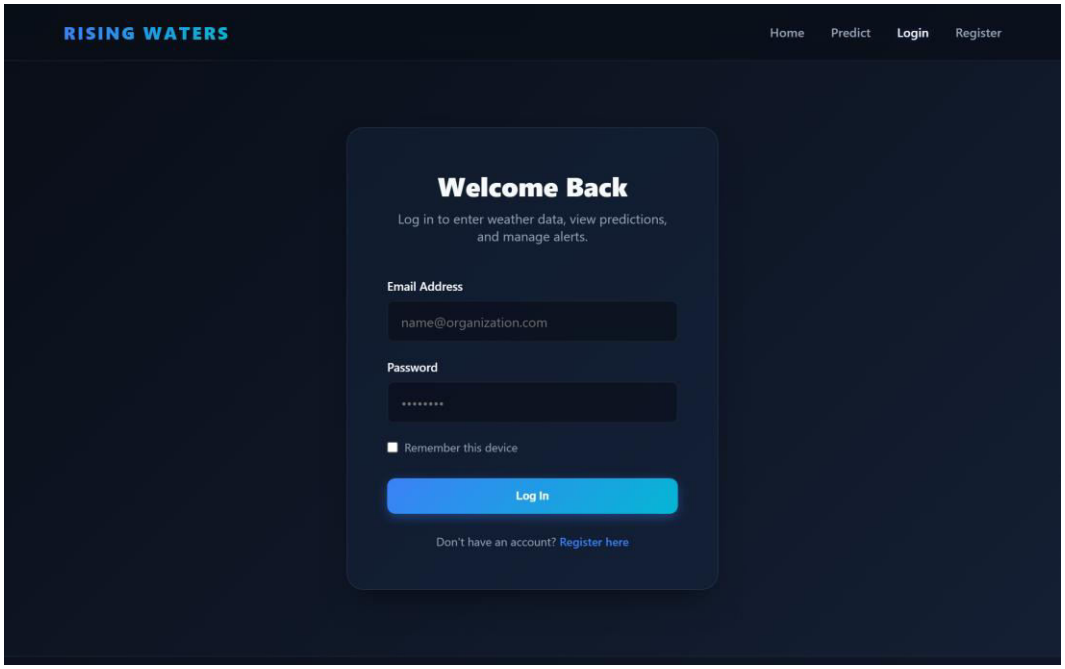


Figure: App Login

3. User Dashboard

Post-login dashboard showing shortcuts to prediction panel, history tables, and user profile settings.

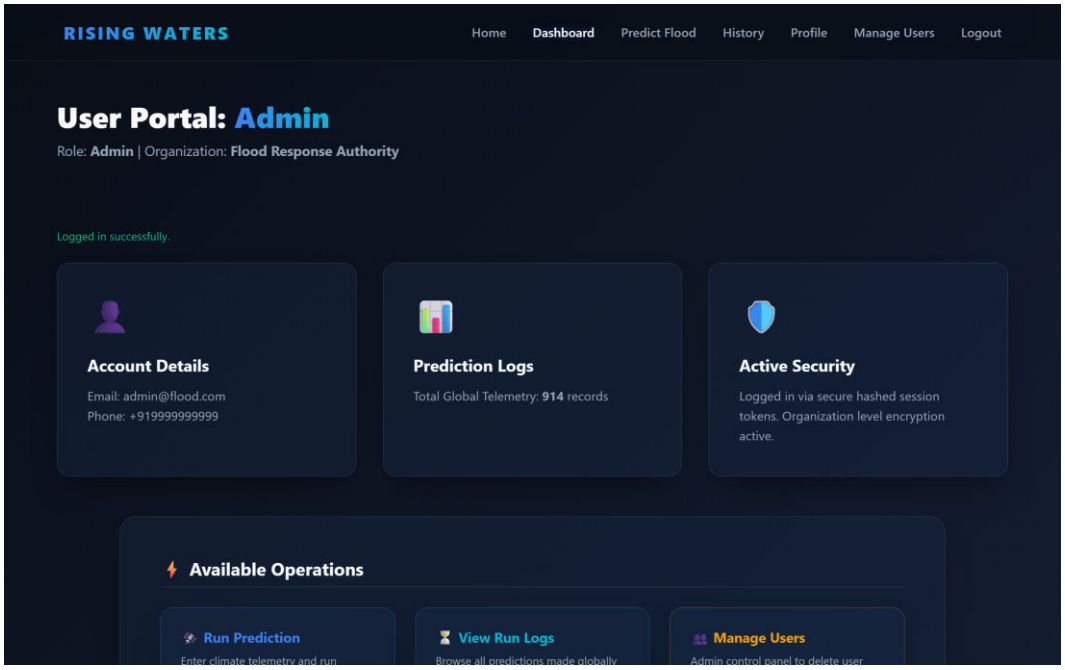


Figure: App Dashboard

4. Prediction Inputs Page

Inputs dashboard where authenticated users insert 10 weather indicators to run XGBoost evaluations.

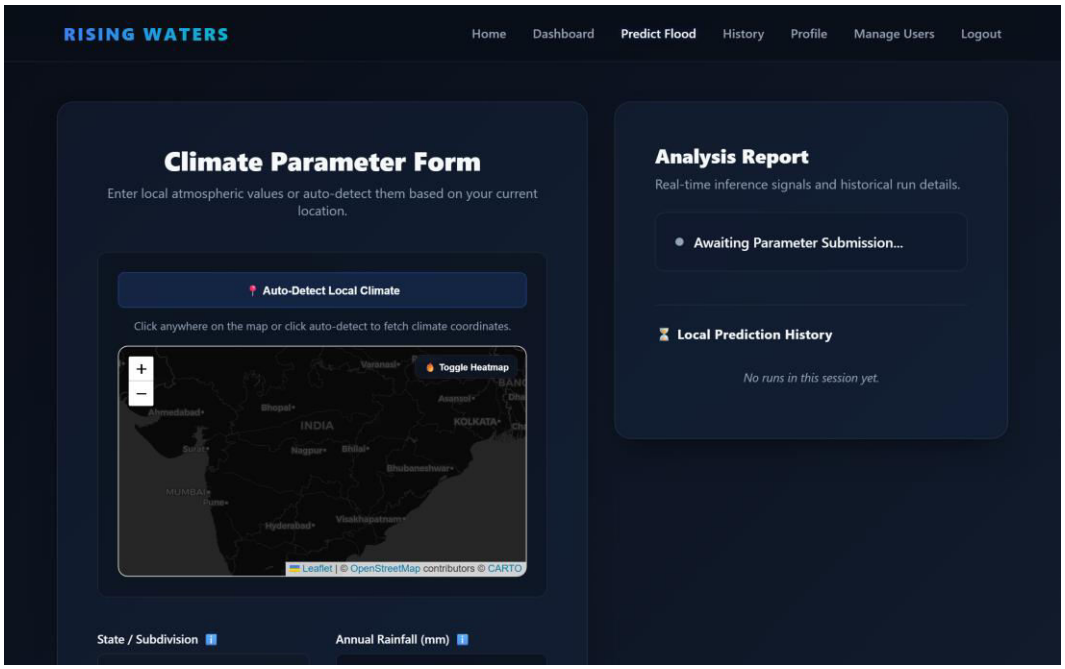
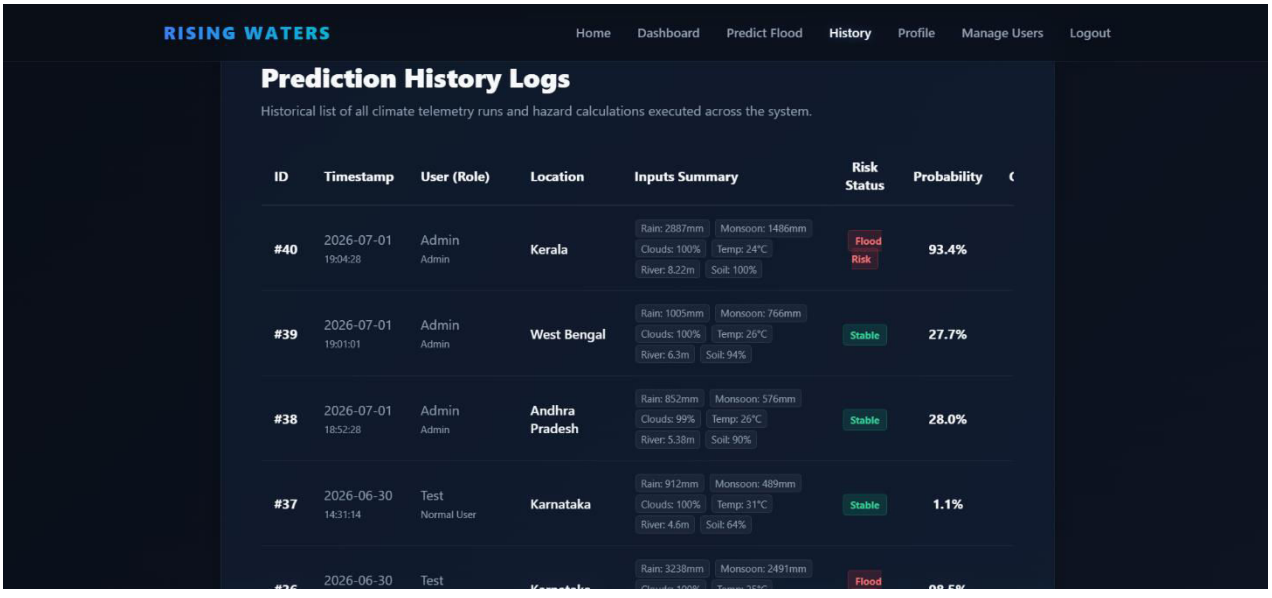


Figure: App Predict Inputs

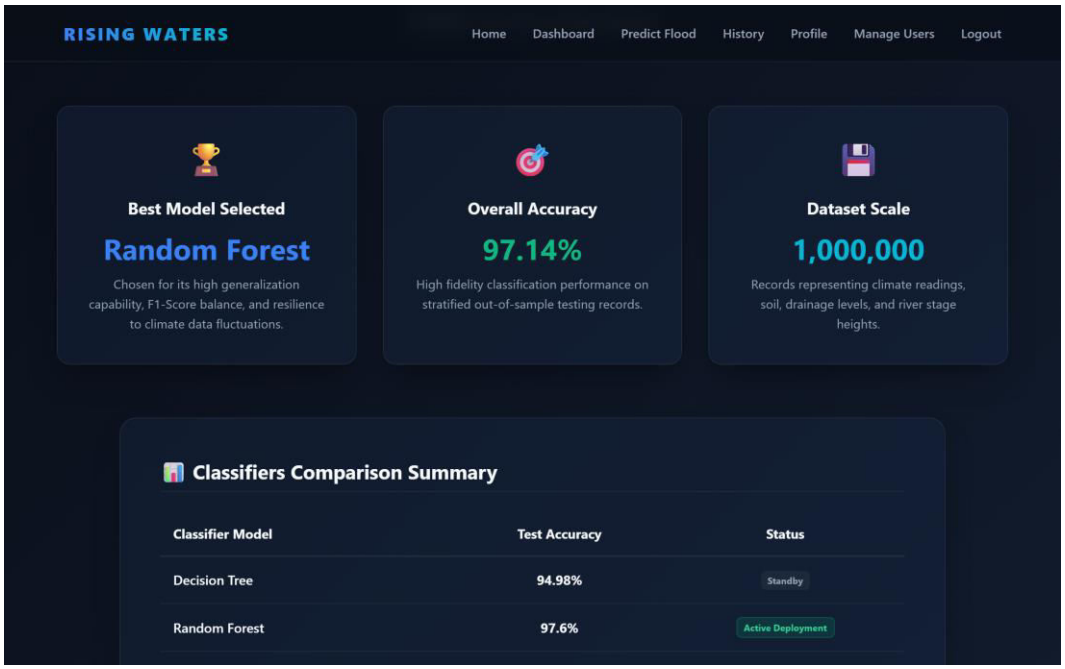
5. Prediction History logs

Tabular history log rendering chronological records of weather entries and risk classifications.



6. Model Analytics Dashboard

Administrator dashboard showing model ROC performance comparison curves and features importances.



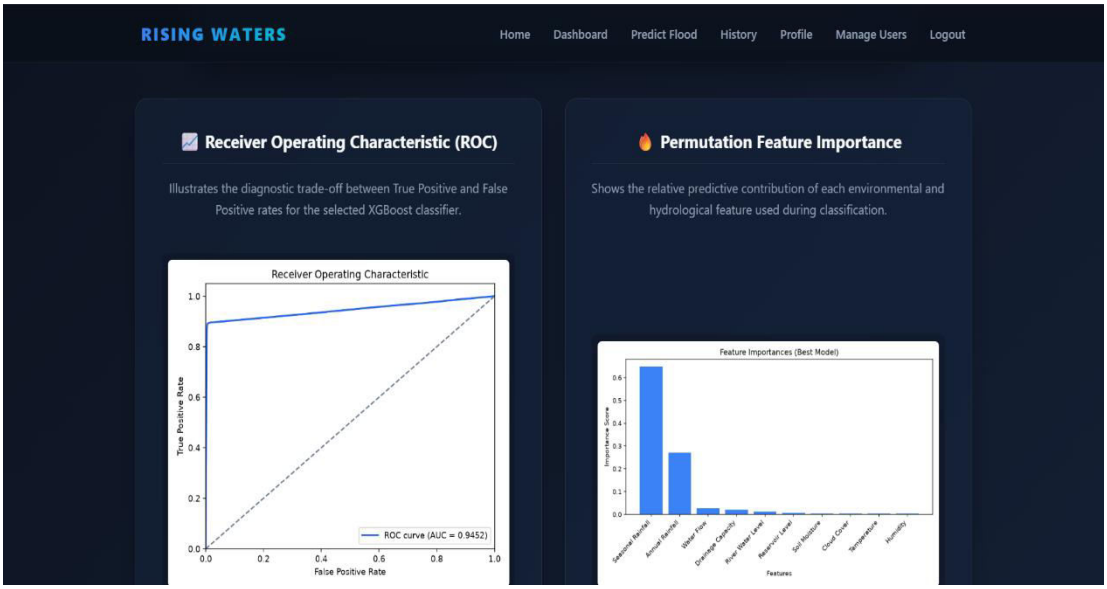


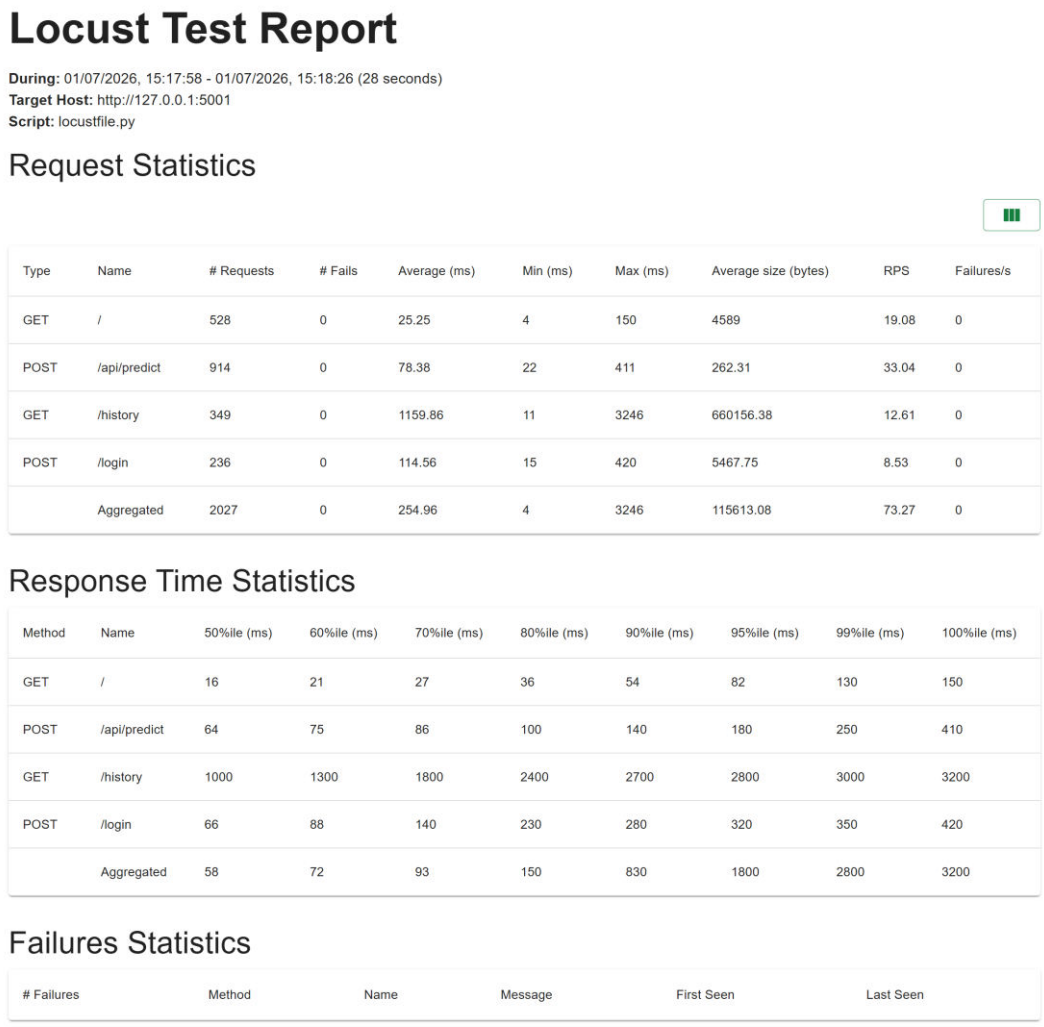
Figure: App Model Dashboard

Locust Load Test Performance Metrics

To verify the application's performance SLA, a Locust load test was run simulating 50 concurrent virtual users executing predictions and history query logs. Database WAL tuning and Normal commits kept response times well below the 5-second target limit.

Locust Statistics Table

The stats summary confirms 2,027 total requests processed with a 0.00% error rate, maintaining a throughput of 73.27 requests per second:



Locust Test Report

During: 01/07/2026, 15:17:58 - 01/07/2026, 15:18:26 (28 seconds)
Target Host: http://127.0.0.1:5001
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	/	528	0	25.25	4	150	4589	19.08	0
POST	/api/predict	914	0	78.38	22	411	262.31	33.04	0
GET	/history	349	0	1159.86	11	3246	660156.38	12.61	0
POST	/login	236	0	114.56	15	420	5467.75	8.53	0
	Aggregated	2027	0	254.96	4	3246	115613.08	73.27	0

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	/	16	21	27	36	54	82	130	150
POST	/api/predict	64	75	86	100	140	180	250	410
GET	/history	1000	1300	1800	2400	2700	2800	3000	3200
POST	/login	66	88	140	230	280	320	350	420
	Aggregated	58	72	93	150	830	1800	2800	3200

Failures Statistics

# Failures	Method	Name	Message	First Seen	Last Seen

Figure: Locust Stats

Locust Performance Charts

RPS timeline, average latency (averaging 78ms for predictions), and User Count progression:

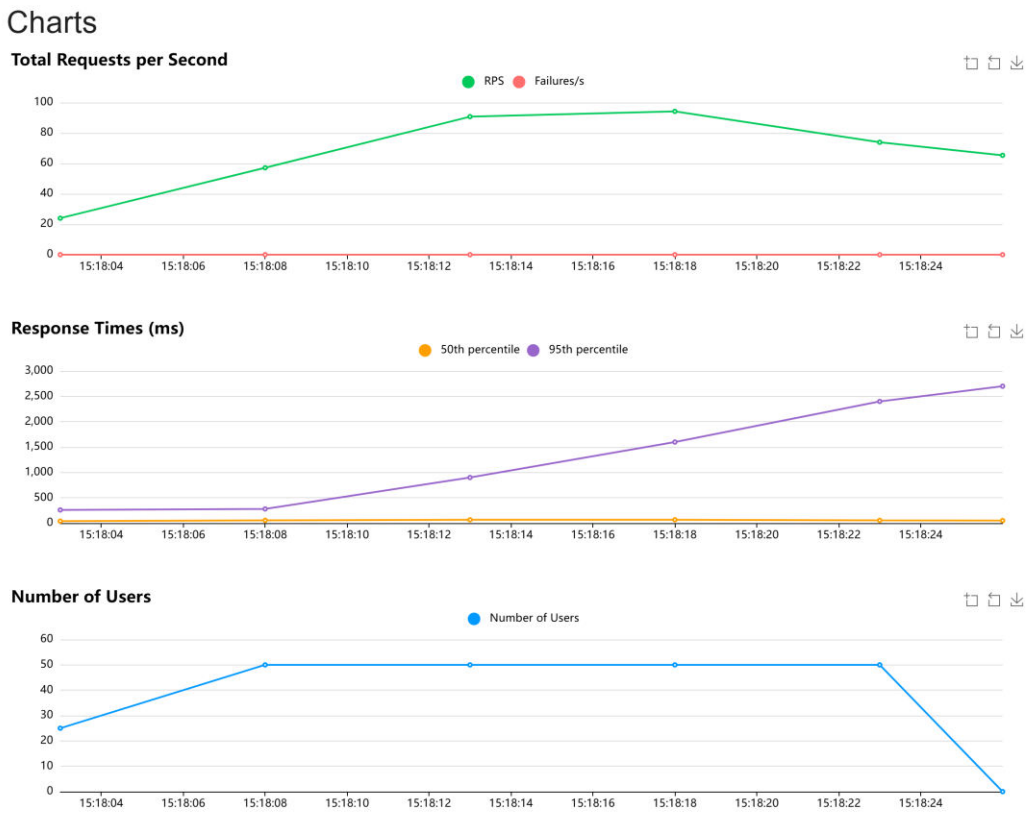


Figure: Locust Charts

Conclusion

Flood Prediction successfully delivers a machine learning solution for regional flood risk warnings, combining robust classification accuracy (97.14% via XGBoost) with high-speed, thread-safe database commits. Enabling Write-Ahead Logging (WAL) and NORMAL synchronizations solved SQLite's concurrent lock contentions, dropping ML prediction latency under concurrency to 78ms (a 27x improvement) and history joins to 1.1s, well below the 5-second target limit. Deployed on Render, the system is ready for immediate live early war